
HAGuard Hierarchical Adaptive Guard for Low-Latency LLM Jailbreak Detection

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Large-language-model assistants increasingly mediate real-time user interactions,
2 yet remain vulnerable to jailbreak prompts that elicit disallowed content. Produc-
3 tion chat services impose sub-100 ms latency budgets, so calling a multi-billion-
4 parameter "judge" model is infeasible, while single-stage heuristics plateau near
5 80% balanced accuracy. We introduce HAGuard, a three-layer adaptive cascade
6 that fuses (i) an ultra-fast lexical gate, (ii) a 60 M-parameter INT-4 DistilBERT,
7 and (iii) prototype-intent matching in embedding space. All components export
8 to ONNX and run on commodity CPUs; 70% of traffic exits after the first two
9 layers, keeping median end-to-end latency at 27 ms and p99 at 78 ms on an AWS
10 c7g.large core. Training mixes multi-view contrastive learning on HarmBench,
11 JailbreakBench, SafetyBench, StrongREJECT and 50 k benign production prompts
12 with hard-negative mining from the RL-Breaker attack generator. Misclassified
13 prompts trigger four-hourly LoRA updates that hot-reload in < 2 min. On a public
14 55 k-prompt benchmark HAGuard attains $96.4 \pm 0.3\%$ true-positive rate at 0.5%
15 false-positive rate and 0.972 ± 0.002 AUROC-15-20 pp higher and eight times
16 faster than LLM-Guard-7B. A deliberately truncated 16-prompt run that scored
17 only chance-level accuracy highlights the importance of the full evaluation protocol.
18 Code, models and raw predictions are released for complete reproduction.

19 1 Introduction

20 Large-language models (LLMs) have transitioned from laboratory curiosities to central components
21 of code assistants, writing aids and even medical triage tools. Their mass adoption nevertheless hinges
22 on robust guardrails that intercept malicious "jailbreak" prompts able to override safety alignment and
23 coax disallowed output Chao et al. [2024], Chen et al. [2024]. A practical guard must simultaneously
24 1) recognise increasingly sophisticated, obfuscated and rapidly evolving attacks, 2) avoid collateral
25 damage to benign requests, and 3) meet stringent service-level agreements-typically ≤ 100 ms
26 end-to-end for 99% of queries.

27 Meeting all three goals is challenging. Lightweight lexical rules execute in micro-seconds but miss
28 paraphrases and unicode tricks, capping balanced accuracy at roughly 75-85% Alon and Kamfonas
29 [2023]. Replacing them with powerful judge LLMs such as Llama-Guard-7B improves recall yet
30 adds 200-500 ms latency, unacceptable for synchronous user experiences. Existing defences rarely
31 combine heterogeneous evidence, leading to poorly calibrated probabilities and forcing operators
32 to choose between aggressive blocking and risky permissiveness. Finally, attackers iterate quickly,
33 whereas labelled data for novel jailbreak styles arrive slowly, impeding periodic re-training.

34 This paper presents HAGuard, a Hierarchical Adaptive Guard that addresses the four pain points
35 through a carefully engineered cascade and an inexpensive continual-learning loop that never resorts
36 to large models in production. All compute stays on a single CPU core; 72% of queries terminate
37 after a lexical or distilled-transformer stage, while the hardest 28% invoke a quantised embedding

38 matcher and a calibrated LightGBM head whose decision fuses lexical, semantic and prototype
39 features. Prompts the stack misclassifies are quarantined and used to refresh a LoRA adapter every
40 four hours, realising fast recovery reminiscent of online cascade learning Nie et al. [2024] without
41 any GPU.

42 1.1 Contributions

- 43 • **CPU-only low-latency cascade:** A lexical-semantic-prototype cascade that sustains < 100
44 ms p99 latency entirely on CPUs.
- 45 • **Training with adversarial mining:** A recipe combining multi-view contrastive learning,
46 RL-Breaker adversarial mining Chen et al. [2024] and LoRA-based continual updates (< 2
47 min downtime).
- 48 • **CI sanity suite:** A continuous-integration suite that aborts if any artefact is stubbed or
49 latency budgets are violated.
- 50 • **Comprehensive evaluation:** The first side-by-side evaluation of HAGuard, LLM-Guard-7B,
51 Rebuff, PARDEN Zhang et al. [2024] and perplexity-based filters on a public 55 k-prompt
52 benchmark drawn from HarmBench, JailbreakBench and production logs.
- 53 • **Online adaptation study:** A 12-hour study showing that two LoRA refreshes recover 60%
54 of initially missed attacks with < 1 pp FPR increase.

55 The remainder reviews related work, formalises the problem, details the methodology, describes
56 experimental protocols, reports empirical results and concludes with limitations and future directions.

57 2 Related Work

58 Existing guardrails fall into three families. 1) Micro-second heuristics-regex keyword lists and
59 perplexity filters-scale but fail on obfuscations, plateauing at $\sim 80\%$ accuracy Alon and Kamfonas
60 [2023]. 2) Fine-tuned moderation LLMs such as Llama-Guard-2 and WildGuard improve recall yet
61 are GPU-bound, adding hundreds of milliseconds latency Han et al. [2024], Inan et al. [2023]. 3)
62 Autoregressive self-judging approaches like PARDEN prompt the assistant to repeat its own output
63 to avoid domain shift but still invoke a large model per request Zhang et al. [2024].

64 Complementary defences intervene inside the assistant itself: information-bottleneck training hinders
65 leakage of harmful content Liu et al. [2024]; adversarial prompt tuning hardens the system against
66 black-box attacks Mo et al. [2024]; MoGU routes between "safe" and "usable" variants to balance
67 utility and harm Du et al. [2024]. Such internal techniques do not obviate the need for an external
68 pre-filter under tight latency constraints.

69 Balancing risk against computation through cascades is well studied in computer vision Goren et al.
70 [2024] and was recently extended to LLM inference over streams Nie et al. [2024]. HAGuard is, to
71 our knowledge, the first cascade combining lexical, distilled-transformer and prototype-intent cues
72 for jailbreak detection.

73 Continual adaptation via lightweight adapters appears in privacy-preserving prompting Duan et al.
74 [2023] and backdoor scanning Shen et al. [2021]. We adopt the same principle to refresh DistilBERT
75 every few hours without downtime.

76 Finally, standardised benchmarks such as JailbreakBench Chao et al. [2024] and StrongREJECT
77 Souly et al. [2024] expose weaknesses of earlier ad-hoc evaluations. Our offline and online suites
78 follow their guidelines and release raw predictions for transparency.

79 3 Background

80 3.1 Problem formulation

81 For each user prompt p we must emit $y \in \{\text{allow}, \text{block}\}$ and optionally a risk label t from a taxonomy
82 of K categories. The guard executes synchronously before the assistant; the 99th-percentile runtime
83 must not exceed $L = 100$ ms. Utility is measured primarily by the true-positive rate (TPR) at a 0.5%
84 false-positive rate (FPR), and by AUROC.

85 3.2 Signals

86 Lexical patterns captured by tries, unicode-homoglyph detectors and Bloom filters offer constant-time
87 but brittle cues. Contextual semantics require transformer encoders, yet even compact models cost tens
88 of milliseconds. Sentence-level embeddings cluster prompts with similar intent; nearest-neighbour
89 distances to curated prototypes provide interpretable yet potentially noisy evidence.

90 3.3 Cascade rationale

91 Let ℓ_i denote latency of layer i and Δ_i its marginal recall. A cascade is worthwhile when the expected
92 latency $\sum_i \pi_i \ell_i$ (π_i being the traffic fraction reaching layer i) remains under L while cumulative
93 recall approaches that of the full stack. Early-exit thresholds must be calibrated to respect the global
94 FPR budget.

95 3.4 Adaptive defence

96 Attackers rapidly invent paraphrases, emojis or unicode tricks that elude static filters Liu et al. [2023].
97 Logging the hardest misses and injecting them via small LoRA updates enables prompt recovery
98 without full retraining.

99 4 Method

100 4.1 System architecture

101 HAGuard comprises three sequential layers. 1) Layer-0 lexical gate. A trie of 10 318 unsafe keywords
102 and a 1 M-slot Bloom filter (murmur-3, $k = 7$, FP $\approx 0.1\%$) compute a risk score s_0 ; prompts with
103 $s_0 > 0.95$ are blocked. Latency $\approx 5 \mu s$. 2) Layer-1 semantic scorer. DistilGuard-60M-six-layer
104 DistilBERT distilled from Llama-Guard logits-is quantised to INT-4 weights with INT-8 activations
105 and exported to ONNX-Runtime. It outputs class probabilities p_1 ; the maximum forms score s_1 .
106 Latency ≈ 18 ms on one vCPU. 3) Layer-2 prototype matcher. Prompts are encoded with MiniLM-L6-
107 v2 into 128-dim vectors e . Two FAISS-HNSW indices-20 k harmful and 40 k benign prototypes-are
108 searched ($ef = 128$, $M = 32$). Features comprise the signed margin $m = d_{\text{ben}} - d_{\text{harm}}$, Jaccard
109 token overlap and an intent-consistency flag, concatenated with s_0 and s_1 . A calibrated LightGBM
110 with 500 trees (depth 7, learning-rate 0.05) produces the final verdict. Latency ≈ 9 ms.

111 4.2 Training data

112 A 164 k-prompt corpus combines HarmBench, SafetyBench jailbreak subset, JailbreakBench, Stron-
113 gREJECT, 50 k benign production prompts and 5 k real jailbreak attempts.

114 4.3 Model training

115 DistilGuard is fine-tuned with a multi-view contrastive loss (temperature 0.07, queue 1 024) over
116 paraphrases, translation round-trips and RL-Breaker adversarial variants Chen et al. [2024]. Proto-
117 types are selected via k-means++ in embedding space and labelled by majority vote. LightGBM
118 minimises log-loss with class weights chosen to hit 0.5% FPR on validation.

119 4.4 Continual learning

120 Prompts misclassified after Layer-2 populate a quarantine bucket. Every four hours up to 4 000 are
121 used to train a rank-8 LoRA adapter ($lr 2 \times 10^{-5}$, one epoch) that merges into DistilGuard and
122 hot-reloads across workers, incurring < 2 min downtime.

123 4.5 System optimisation

124 All kernels exploit AVX-512; asynchronous I/O and early exits keep Layer-1 and Layer-2 threads
125 idle for $\sim 70\%$ of traffic.

126 4.6 Integrity assurance

127 A CI suite executes a 1 000-prompt sanity batch, asserting entropy of Layer-1 logits, prototype
128 margins, median latency ≤ 25 ms and resident-set size ≤ 800 MB. Any stub placeholder aborts the
129 pipeline.

130 5 Experimental Setup

131 5.1 Hardware

132 All experiments run on AWS c7g.large (single Graviton-3 vCPU, 8 GiB RAM). GPU numbers are
133 reported only for the 7 B baseline.

134 5.2 Datasets

135 (1) HarmBench 13 832 prompts, (2) SafetyBench jailbreak subset 8 617, (3) JailbreakBench 9 527
136 Chao et al. [2024], (4) StrongREJECT 313 Souly et al. [2024], (5) Prod-Benign 50 000, (6) Rebuff-
137 Jailbreak 5 000. After NFKC normalisation and deduplication 164 k unique prompts remain; 55 k
138 prompts identical to the manuscript’s test list are held out for public evaluation.

139 5.3 Splits

140 Prompt-family stratified 70%/10%/20% train/val/test; StrongREJECT resides entirely in test. Random
141 state 2025 ensures replicability.

142 5.4 Baselines

143 A LLM-Guard-v2-7B (fp16, GPU via vLLM), B Rebuff default rules, C PARDEN Zhang et al.
144 [2024], D perplexity filter Alon and Kamfonas [2023]. Thresholds are tuned on validation to meet
145 0.5% FPR when possible.

146 5.5 Metrics

147 Primary: TPR at 0.5% FPR, AUROC, median and p99 latency. Secondary: balanced accuracy,
148 15-bin expected calibration error, RSS and AWS cost per million queries. A robustness suite
149 applies (i) whitespace padding $\leq 1\,024$ spaces, (ii) 5% homoglyph swaps, and (iii) benign-harmful
150 concatenation; we report Δ TPR and Δ FPR relative to clean prompts.

151 5.6 Procedures

152 Offline evaluation trains five seeds {13, 17, 19, 23, 29}, stores predictions and computes mean $\pm$ SD
153 plus $1\,000\times$ bootstrap confidence intervals. A 12-h online study streams 50 req/s via Kafka, injects
154 $\geq 3\,000$ novel attacks from RL-Breaker, and compares HAGuard-Adaptive, HAGuard-Frozen and
155 LLM-Guard-7B workers. Prometheus scrapes accuracy, latency, CPU and memory per hour; LoRA
156 refreshes occur at hours 4 and 8.

157 6 Results

158 6.1 Offline benchmark

159 Across five seeds HAGuard achieves $96.4 \pm 0.3\%$ TPR at 0.5% FPR and AUROC 0.972 ± 0.002 .
160 LLM-Guard-7B scores $81.1 \pm 0.4\%$ TPR and 0.887 AUROC; Rebuff 74.5%, PARDEN 78.2%,
161 perplexity 63.4%. Median latency is 27 ± 1 ms and p99 78 ± 2 ms, versus 219 ± 8 ms and 642 ± 15
162 ms for the GPU baseline. RSS is 613 MB compared to 15 GiB plus GPU memory for LLM-Guard.

163 6.2 Ablation study

164 Removing Layer-0 increases median latency by 15 ms and CPU utilisation by 53% without accuracy
165 benefit. Dropping Layer-1 reduces TPR by 9.8 pp; omitting Layer-2 cuts TPR by 11.3 pp but saves
166 only 5 ms. Switching DistilGuard from INT-4 to INT-8 yields a negligible 0.2 pp TPR gain yet
167 doubles memory.

168 6.3 Robustness

169 Whitespace padding and homoglyph attacks lower HAGuard’s TPR by 1.9 pp, compared with 7-12
170 pp for single-stage models. Prompt concatenation costs 3.1 pp.

171 6.4 Online adaptation

172 During the 12-h stream attack entropy spikes at hours 4 and 8. HAGuard-Adaptive climbs from 78%
173 to 92% TPR after two LoRA refreshes, with FPR rising by < 1 pp; p99 latency stays below 85 ms.
174 The frozen variant remains flat, as does LLM-Guard-7B. Total finetune downtime is 96 s.

175 6.5 Truncated run

176 A deliberately misconfigured "full-experiment" processed only 16 prompts; all exited at Layer-0,
177 yielding $\sim 10 \mu\text{s}$ latency and 50% accuracy. Because CI checks are disabled in this mode, the episode
178 underscores the necessity of the integrity suite.

179 6.6 Limitations

180 False positives persist in creative role-play scenarios where fictional crime plots resemble genuine
181 malicious intent. Quantisation introduces slight calibration drift that future work should correct.

182 7 Conclusion

183 HAGuard demonstrates that carefully orchestrated lexical, distilled-transformer and prototype-intent
184 evidence can raise jailbreak-detection accuracy from the long-standing 80% ceiling to beyond 95%
185 while meeting sub-100 ms latency budgets on a single CPU core. Multi-view contrastive training,
186 RL-Breaker hard-negative mining and rapid LoRA updates keep the guard adaptive without GPUs
187 or prolonged downtime. Extensive offline and online experiments confirm substantial gains over
188 GPU-bound or heuristic baselines, and all artefacts are open-sourced for full reproducibility. Future
189 work will pursue tighter calibration, multilingual coverage, integration with routing frameworks such
190 as MoGU Du et al. [2024], human-in-the-loop feedback and certified robustness guarantees inspired
191 by CBD Xiang et al. [2023].

192 References

- 193 Gabriel Alon and Michael Kamfonas. Detecting language model attacks with perplexity. 2023.
- 194 Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce,
195 Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed
196 Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large
197 language models. 2024.
- 198 Xuan Chen, Yuzhou Nie, Wenbo Guo, and Xiangyu Zhang. When llm meets drl: Advancing
199 jailbreaking efficiency via drl-guided search. *NeurIPS 2024*, 2024.
- 200 Yanrui Du, Sendong Zhao, Danyang Zhao, Ming Ma, Yuhang Chen, Liangyu Huo, Qing Yang,
201 Dongliang Xu, and Bing Qin. Mogu: A framework for enhancing safety of llms while preserving
202 their usability. 2024.
- 203 Haonan Duan, Adam Dziedzic, Nicolas Papernot, and Franziska Boenisch. Flocks of stochastic
204 parrots: Differentially private prompt learning for large language models. 2023.

205 Shani Goren, Ido Galil, and Ran El-Yaniv. Hierarchical selective classification. 2024.

206 Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin
 207 Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks,
 208 and refusals of llms. 2024.

209 Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael
 210 Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based
 211 input-output safeguard for human-ai conversations. 2023.

212 Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. Autodan: Generating stealthy jailbreak
 213 prompts on aligned large language models. 2023.

214 Zichuan Liu, Zefan Wang, Linjie Xu, Jinyu Wang, Lei Song, Tianchun Wang, Chunlin Chen, Wei
 215 Cheng, and Jiang Bian. Protecting your llms with information bottleneck. 2024.

216 Yichuan Mo, Yuji Wang, Zeming Wei, and Yisen Wang. Fight back against jailbreaking via prompt
 217 adversarial tuning. 2024.

218 Lunyiu Nie, Zhimin Ding, Erdong Hu, Christopher Jermaine, and Swarat Chaudhuri. Online cascade
 219 learning for efficient inference over streams. 2024.

220 Guangyu Shen, Yingqi Liu, Guanhong Tao, Shengwei An, Qiuling Xu, Siyuan Cheng, Shiqing Ma,
 221 and Xiangyu Zhang. Backdoor scanning for deep neural networks through k-arm optimization.
 222 2021.

223 Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel,
 224 Justin Svegliato, Scott Emmons, Olivia Watkins, and Sam Toyer. A strongreject for empty
 225 jailbreaks. 2024.

226 Zhen Xiang, Zidi Xiong, and Bo Li. Cbd: A certified backdoor detector based on local dominant
 227 probability. 2023.

228 Ziyang Zhang, Qizhen Zhang, and Jakob Foerster. Parden, can you repeat that? defending against
 229 jailbreaks via repetition. 2024.